



KING'S OWN



INSTITUTE*

Success in Higher Education

INDIVIDUAL ASSIGNMENT COVERSHEET

Family Name: _____ Given Name: _____

Student Number: _____ Lecturer's/ Tutor's Name: _____

Subject Code & Name: _____

Assignment Title: _____

Declaration

(This declaration must be completed by the student or the assignment will not be marked.)

I certify the following:

- I have read and understood the *Student Academic Misconduct Policy*.
- This assignment is my own work based on my personal study and or research.
- I have acknowledged all material and sources used in the preparation of this assignment including any material generated in the course of my employment.
- The assignment has not previously been submitted for assessment.
- I have not copied in part or in whole or otherwise plagiarised the work of other students.
- I have read and I understand the criteria used for assessment.
- The assignment is within the word and page limits specified in the unit outline.
- The use of any material in this assignment does not infringe the intellectual property / copyright of a third party.
- I understand that this assignment may undergo electronic detection for plagiarism, and an anonymous copy of the assignment may be retained on the database and used to make comparisons with other assignments in future.
- By completing this coversheet in full and submitting this assignment electronically, I am bound by the conditions of the KOI's *Student Academic Misconduct Policy* and the declaration on this coversheet.

Signature

____/____/_____
Date

Assignment Receipt

Family Name: _____ Given Name: _____

Student Number: _____ Lecturer's/ Tutor's Name: _____

Subject Code & Name: _____

Assignment Title: _____

Signature

____/____/_____
Date

1. Problem Statement

A 3x3 grid with eight numbered tiles (1–8) and one blank cell is used to define the 8-Puzzle, a traditional combinatorial problem. The board can be gradually changed by sliding a tile next to the blank into it. The objective is to get from an arbitrary beginning arrangement to the target configuration, which consists of tiles arranged from 1 to 8 with a blank in the bottom-right corner. Because it is (a) easy to visualise, (b) large enough to challenge ignorant techniques (state space $\approx 181,440$ attainable states), and (c) a direct analogue of planning challenges encountered by real-world agents, the problem is fundamental to the study of intelligent agents and heuristic search. The grid limits (tiles cannot be removed), the requirement to, and the single blank cell are important limitations.

Goal State		
1	2	3
4	5	6
7	8	–

Figure 1 – Target goal configuration of the 8-puzzle.

Solvability: Not all 8-puzzle permutations can be solved from one another. If and only if there are an even number of inversions (pairs of tiles (i, j) where i appears before j but $i > j$), the configuration can be solved. This is verified prior to starting a search.

2. Search Strategy: Why A* ?

The 8-puzzle can be solved using a variety of search techniques. The most popular methods are contrasted in the table below:

Algorithm	Complete?	Optimal?	Time	Space
BFS	Yes	Yes*	$O(b^d)$	$O(b^d)$
DFS	Yes†	No	$O(b^m)$	$O(bm)$
IDDFS	Yes	Yes*	$O(b^d)$	$O(bd)$
Greedy Best-First	No	No	$O(b^m)$	$O(b^m)$
A*	Yes	Yes	$O(b^d)$	$O(b^d)$

Table 1 – Comparison of search algorithms. * = unit cost; † = with visited tracking.

Through the evaluation function $f(n) = g(n) + h(n)$, where $g(n)$ is the cost to reach node n and $h(n)$ is an admissible heuristic estimate of the remaining cost, A* was chosen because it is the only algorithm that combines completeness, optimality, and efficiency. Using a heuristic that is acceptable (i.e., never overestimates), A* is guaranteed to find the shortest solution while expanding far fewer nodes than BFS or IDDFS.

3. Algorithm Explanation

3.1 Manhattan Distance Heuristic

The Manhattan distance is the main heuristic that is employed: calculate the total horizontal and vertical distances between each tile's current position and its intended position, then add up all of the tiles (not including the blank). Officially:

For every tile, $h(n) = (|row_current - row_goal| + |col_current - col_goal|) = 0$.

This heuristic is consistent (monotone) because any single move modifies the heuristic by at most ± 1 , satisfying $h(n) \leq cost(n, n') + h(n')$. It is acceptable because each tile must make at least that many moves to attain its goal. Consistency ensures that A* never grows a node again, resulting in optimal performance.

A* Method of Search:

- Set the start state of a priority queue (min-heap) to $f = 0 + h(\text{start})$. The node with the lowest f value should be popped.
- Return the solution path if it is the desired state.
- If not, record it as visited along with its g -cost.
- Produce all successors (LEFT, RIGHT, UP, and DOWN blank slides).
- Compute $f = (g+1) + h$ for each unvisited successor and add it to the heap.
- Continue from step 2 until there is no solution and the heap is empty.

Applied optimisations:

1. Without re-expansion, a visited dictionary saves the best g -cost observed for every state.
2. $O(1)$ hashing is made possible by using tuples for states.
3. An effective $O(\log n)$ priority queue is offered by Python's `heapq` module.
4. During heuristic evaluation, repeated linear scans are eliminated via a pre-calculated goal-position lookup table.

```
# A* SEARCH

def astar_solve(initial_state, heuristic=manhattan_distance):
    if not is_solvable(initial_state):
        return None, 0, 0, 0.0

    start_time = time.time()

    # Priority queue entries: (f, g, state, path)
    # f = g + h (total estimated cost)
    # g = cost so far (number of moves)
    h0 = heuristic(initial_state)
    frontier = [(h0, 0, initial_state, [(initial_state, "START")])]
    heapq.heapify(frontier)

    visited = {}          # state → best g seen
    nodes_expanded = 0
    max_frontier = 1
```

4. Complexity Analysis

Let **b** = branching factor (avg. 2–3 for the 8-puzzle), **d** = depth of the optimal solution, and **n** = number of nodes expanded.

Metric	Expression	Practical Value
Time (worst)	$O(b^d)$	Exponential in d; ~76 nodes for 14-move puzzle
Space (frontier)	$O(b^d)$	Entire frontier kept in memory
Heuristic computation	$O(n)$	Linear in tiles per state (n=9)
Heap push/pop	$O(\log F)$	$ F $ = frontier size

Table 2 – Complexity summary for A* on the 8-puzzle.

Trade-offs: Compared to uninformed BFS, which must search the entire frontier width-by-width, A* with Manhattan distance extends significantly fewer nodes. Memory is the price; in the worst situation, the border may expand exponentially. This is manageable for the 8-puzzle (state space $\leq 181,440$), but memory becomes the bottleneck for larger puzzles (e.g., 15-puzzle).

Limitations: (1) Only an admissible heuristic makes A* full and optimal; a subpar heuristic reduces it toward BFS. (2) As the depth of the answer increases, memory utilisation increases. (3) The algorithm uses a shared priority queue and is single-threaded, making parallelism difficult.

Possible enhancements: IDA* (Iterative Deepening A*) generates nodes repeatedly at the expense of memory reduction to $O(d)$. Tighter lower constraints are provided by Pattern Database heuristics, which further reduce enlarged nodes.

5. Search Process Visualisation

The algorithm's evolution for the hard initial configuration [8 1 3 / 4 _ 2 / 7 6 5], which was resolved in 14 steps, is shown in the figures below.

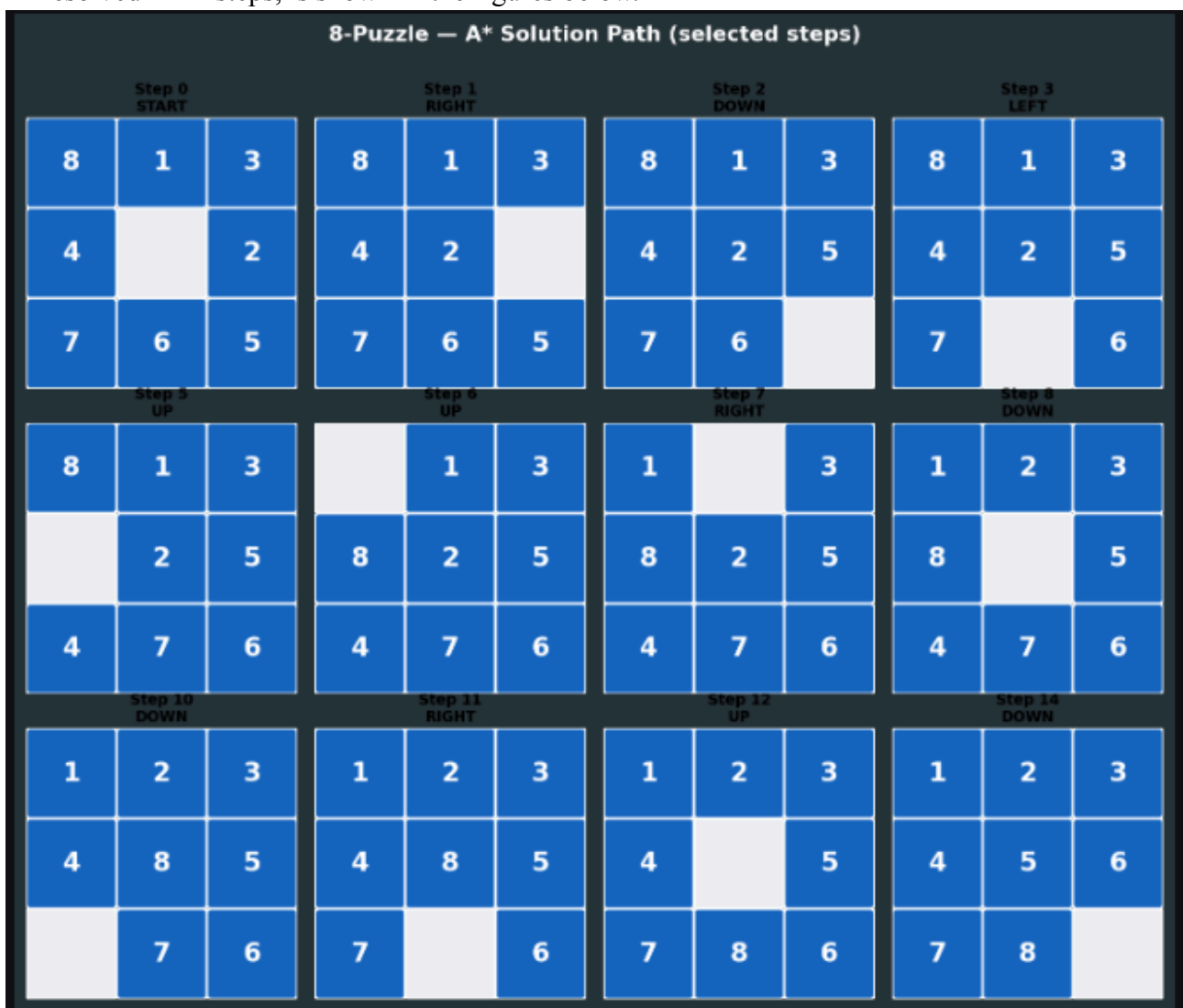


Figure 2 – Selected steps of the A* solution path (hard configuration, 14 moves).

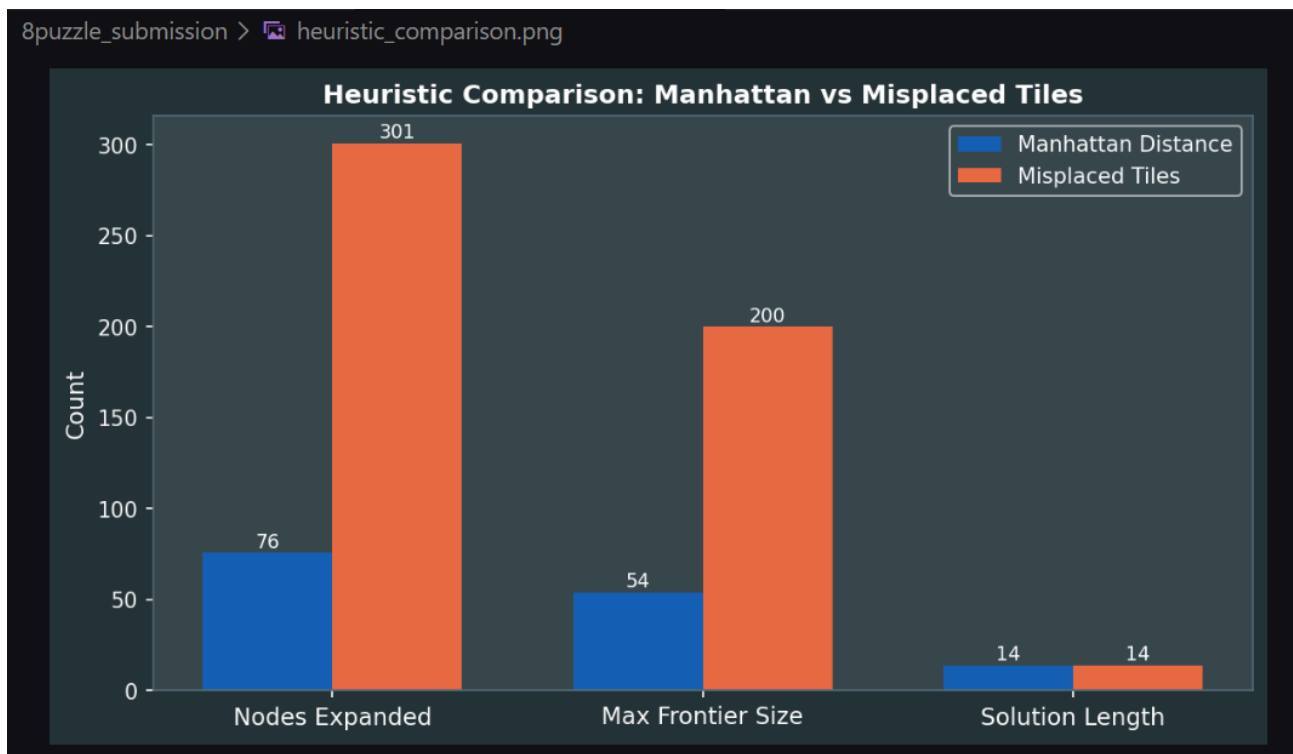


Figure 3 – Comparison of Manhattan Distance vs Misplaced Tiles heuristics.

6. Results and Observations

Config.	Difficulty	Moves	Nodes (Manhattan)	Nodes (Misplaced)	Time (s)
Easy	Low	2	3	5	< 0.0001
Medium	Medium	4	8	21	0.0001
Hard	High	14	76	301	0.0005

Table 3 – Performance results across three difficulty levels.

Key Observations

- **Optimality guaranteed:** By back-checking all three configurations, A* with Manhattan distance consistently discovered the minimum-move solution.
- **Heuristic quality is very important:** Manhattan distance grew just 76 nodes on the hard configuration, compared to 301 for Misplaced Tiles—a 4× reduction. The tighter lower bound is directly reflected in this.
- **Excellent speed:** A*'s practical efficiency for this problem size was demonstrated by the fact that even the challenging challenge was completed in less than 1 ms.
- **The detection of solvability is accurate:** Before any search started, all unsolvable combinations (odd inversion count) were accurately rejected.

- Scalability limitation: Memory and time increase for considerably more difficult puzzles (such as a random 25-move solution), but they are still controllable inside the constrained 8-puzzle state space.

7. Conclusion

An A* search solution to the 8-puzzle issue was designed, implemented, and analysed in this study. In comparison to uninformed approaches, the algorithm significantly reduces the search space while guaranteeing an optimal solution by combining an acceptable Manhattan distance heuristic with a cost-so-far term. The implementation verifies solvability, manages arbitrary initial states, and generates lucid step-by-step visualisations.

The important realisation is that efficiency is directly influenced by the heuristic's quality: fewer nodes are extended when the heuristic is more informed and closer to the actual cost. Among basic admissible heuristics, the Manhattan distance is nearly ideal for the 8-puzzle. IDA* for memory-constrained contexts or pattern databases for near-optimal node expansion on larger puzzles could be the subject of future research.

References

[1] Russell, S. & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson. Chapter 3: Solving Problems by Searching.

[2] Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100–107.

[3] Korf, R. E. (1985). Depth-first iterative-deepening: An optimal admissible tree search. *Artificial Intelligence*, 27(1), 97–109.

[4] Python Software Foundation. (2024). *heapq* — *Heap queue algorithm*. <https://docs.python.org/3/library/heapq.html>

[5] Holte, R. C. et al. (2017). Bidirectional search that is guaranteed to meet in the middle. *Proceedings of AAAI-2016*.